École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise

Complements and Tools of Operations Research (CORO)

Aerodrome Race Optimisation

Inspired by the Coupe Breitling 100/24

Student	**Score**
DIN Sokheng	.....

Lecturer: Prof. Eric Soutil

Academic Year 2025–2026

Contents

1 Introduction and Problem Statement **2**
1.1 Graph Construction . . . 2

2 Integer Linear Programming Model 1: MTZ Formulation **3**
2.1 Variables . . . 3
2.2 Objective . . . 3
2.3 Constraints . . . 3
2.4 MTZ Algorithm Overview . . . 4

3 Integer Linear Programming Model 2: Subtour Elimination Constraints (SEC) **5**
3.1 Variables and Base Constraints . . . 5
3.2 Subtour Elimination . . . 5
3.3 Iterative Solve-and-Cut . . . 6
3.4 Why SEC gives a tighter LP relaxation . . . 6

4 Heuristic **7**
4.1 Phase 1: Greedy Construction . . . 7
4.2 Phase 2: 2-opt Local Search . . . 7
4.3 Phase 3: Node Removal . . . 8

5 Computational Results **8**
5.1 Instance `instance_6_1` ($n = 6$, $m = 2$, $R = 6$, $V_{\min} = 4$) . . . 9
5.2 Instance `instance_40_1` ($n = 40$, $m = 2$, $R = 26$, $V_{\min} = 14$) . . . 9
5.3 Instance `instance_80_1` ($n = 80$, $m = 9$, $R = 31$, $V_{\min} = 35$) . . . 10
5.4 Summary . . . 11

6 Custom Instances **11**

7 Analysis and Critical Discussion **13**
7.1 MTZ vs. SEC: Theory Confirmed, Practice More Nuanced . . . 13
7.2 Integrality Gap and Bound Quality . . . 14
7.3 Why the Heuristic Fails on Sparse Instances . . . 14
7.4 Limitations of the Models . . . 15
7.5 Connections to Course Material . . . 15

8 Conclusion **15**

1 Introduction and Problem Statement

This project addresses a combinatorial optimisation problem inspired by the *Coupe Breitling 100/24*, a French aeronautical competition in which pilots fly small aircraft across France, landing at as many aerodromes as possible within 24 hours. Organisers impose geographic regions that must each be visited at least once to ensure spatial coverage of the territory.

We are given n aerodromes numbered $1, \ldots, n$. Each aerodrome i has integer coordinates (x_i, y_i) and belongs to region $r_i \in \{0, 1, \ldots, m\}$, where $r_i = 0$ means no region is assigned. We are also given a starting aerodrome s, an arriving aerodrome t, a minimum number $V_{\min}$ of aerodromes to visit, and a maximum range R (the furthest distance a plane can fly without landing). Distances are computed as the Euclidean distance rounded to the nearest integer:

$$d(i,\, j) \;=\; \text{round}\Big(\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}\Big)\,.$$

The objective is to find a directed path $s = v_1 \to v_2 \to \cdots \to v_k = t$ minimising the total distance $\sum_{\ell=1}^{k-1} d(v_\ell,\, v_{\ell+1})$ subject to:

$$d(v_\ell,\, v_{\ell+1}) \le R \quad \forall\, \ell = 1, \ldots, k{-}1 \qquad \text{(range constraint)}, \tag{1}$$

$$\forall\, \kappa \in \{1, \ldots, m\},\;\; \exists\, \ell : r_{v_\ell} = \kappa \qquad \text{(region coverage)}, \tag{2}$$

$$k \ge V_{\min} \qquad \text{(minimum visits)}. \tag{3}$$

This problem is related to the Travelling Salesman Problem (TSP) studied in the lectures, but differs in several important ways. First, we seek a *path* from s to t, not a Hamiltonian cycle. Second, we select a *subset* of aerodromes to visit rather than visiting all of them. Third, the region-covering constraints (2) impose a set-covering structure that has no direct analogue in the classical TSP. Finally, the range constraint (1) restricts which consecutive pairs are feasible, unlike the complete graph assumption in standard TSP.

1.1 Graph Construction

We construct a directed graph $G = (V, A)$ where $V = \{1, \ldots, n\}$ and $A = \{(i, j) : i \neq j,\; d(i,\, j) \le R\}$. The range constraint (1) is enforced structurally: arcs violating it simply do not exist. This reduces the number of binary variables from $n(n{-}1)$ to $|A|$, which can be significantly smaller when R is tight relative to the coordinate domain.

2 Integer Linear Programming Model 1: MTZ Formulation

The first model uses the Miller–Tucker–Zemlin (MTZ) subtour elimination technique [6], which was presented in the lectures in the context of the asymmetric TSP. The idea is to introduce continuous *position* variables u_i that assign each visited node its order along the path; a subtour is impossible because positions must strictly increase along any sequence of used arcs.

2.1 Variables

- $x_{ij} \in \{0, 1\}$ for each $(i, j) \in A$: arc (i, j) is in the path.
- $y_i \in \{0, 1\}$ for each $i \in V$: aerodrome i is visited.
- $u_i \in [0,\, n]$ for each $i \in V$: position of i along the path (continuous).

2.2 Objective

$$\min \sum_{(i,j)\in A} d(i,\, j)\; x_{ij}.$$

2.3 Constraints

(C1) Source flow. The path starts at s: exactly one arc leaves s, none enters it.

$$\sum_{j:(s,j)\in A} x_{sj} = 1, \qquad \sum_{j:(j,s)\in A} x_{js} = 0.$$

(C2) Sink flow. The path ends at t: exactly one arc enters t, none leaves it.

$$\sum_{j:(j,t)\in A} x_{jt} = 1, \qquad \sum_{j:(t,j)\in A} x_{tj} = 0.$$

(C3) Flow balance at intermediate nodes. For every $i \in V \setminus \{s, t\}$:

$$\sum_{j:(j,i)\in A} x_{ji} = y_i, \qquad \sum_{j:(i,j)\in A} x_{ij} = y_i.$$

A node is visited ($y_i = 1$) if and only if exactly one arc enters it and one leaves it. If $y_i = 0$, no arc touches node i.

(C4) Endpoints visited. $y_s = 1,\ y_t = 1.$

(C5) Region coverage. For each region $\kappa = 1, \ldots, m$:

$$\sum_{i:\, r_i=\kappa} y_i \;\geq\; 1.$$

This is a classical *set covering* constraint: at least one aerodrome from each region must be selected.

(C6) Minimum visits. $\sum_{i \in V} y_i \;\geq\; V_{\min}$.

(C7) MTZ subtour elimination. For all $(i, j) \in A$ with $i \neq s$ and $j \neq s$:

$$u_i - u_j + n\, x_{ij} \;\leq\; n - 1.$$

This is the core of the MTZ formulation. We analyse the two cases:

- If $x_{ij} = 0$: the constraint becomes $u_i - u_j \leq n - 1$, which is always satisfied since $u_i \leq n$ and $u_j \geq 0$. The constraint is inactive.
- If $x_{ij} = 1$: the constraint becomes $u_j \geq u_i + 1$. This forces node j to have a strictly later position than node i in the path.

Since positions strictly increase along every used arc, any subset of nodes forming a directed cycle would require positions to simultaneously increase and return to their starting value, which is impossible. This eliminates all subtours.

As discussed in the lectures on the TSP, the MTZ formulation uses only $O(n^2)$ constraints (polynomial), but its LP relaxation is weaker than the subtour polytope because the continuous u_i variables allow fractional solutions that "look like" subtours to the relaxation. This typically leads to more branch-and-bound nodes compared to the SEC formulation.

(C8) Linking position to visit. $u_s = 0$, and for all $i \neq s$: $u_i \leq n \cdot y_i$.

If node i is not visited ($y_i = 0$), this forces $u_i = 0$, which tightens the LP relaxation by preventing unvisited nodes from taking arbitrary position values.

2.4 MTZ Algorithm Overview

The model is passed to HiGHS, which solves it via branch-and-bound. The pseudocode below summarises the overall resolution procedure.

Algorithm 1 MTZ-based solve

```
Build arc set A = {(i, j) : d(i, j) ≤ R, i ≠ j}.
Formulate ILP with variables x_ij, y_i, u_i and constraints (C1)–(C8).
Solve LP relaxation at root ⇒ lower bound z_LP.
if LP solution is integer then
    return LP solution.
end if
Branch on a fractional x_ij variable (set to 0 or 1).
repeat
    Solve LP at current B&B node.
    if LP infeasible then prune (infeasibility).
    end if
    if LP bound ≥ incumbent z* then prune (bound).
    end if
    if LP solution is integer and cost < z* then update z* ← cost.
    end if
    Otherwise, branch on the most fractional variable.
until B&B queue is empty
return best integer solution found with cost z*.
```

3 Integer Linear Programming Model 2: Subtour Elimination Constraints (SEC)

The second model replaces the MTZ constraints with *subtour elimination constraints* (SEC), following the approach of Dantzig, Fulkerson, and Johnson [1]. This was also covered in the lectures as the stronger formulation for the TSP, where we saw that SEC describes the subtour polytope exactly and yields a tighter LP relaxation than MTZ.

3.1 Variables and Base Constraints

The model uses the same x_{ij} and y_i variables and the same constraints (C1)–(C6) as Model 1. The position variables u_i and constraints (C7)–(C8) are removed entirely.

3.2 Subtour Elimination

For every non-empty subset $S \subseteq V \setminus \{s, t\}$:

$$\sum_{\substack{(i,j)\in A \\ i\in S,\, j\in S}} x_{ij} \;\le\; |S| - 1. \tag{SEC}$$

The reasoning is combinatorial: among $|S|$ nodes, any acyclic subgraph (in particular, a forest) can use at most $|S| - 1$ arcs. If the number of arcs within S reaches $|S|$ or more, a directed cycle must exist within S. Since such a cycle would be disconnected from the s–t path (because $s, t \notin S$), the constraint forbids it.

The number of possible subsets S is 2^{n-2}, so we cannot add all constraints upfront. This is the same situation encountered in the lectures on cutting plane methods: we have an exponential family of valid inequalities and must use a *separation procedure* to identify violated ones on the fly.

3.3 Iterative Solve-and-Cut

Our implementation uses the following iterative approach, which is a simplified version of the cutting plane algorithm from the lectures on Gomory cuts and SEC for the TSP:

Algorithm 2 Iterative SEC generation

```
Solve the ILP with constraints (C1)–(C6) only.
loop
    Extract the integer solution. Build the subgraph of arcs with x_ij = 1.
    BFS from s: find all nodes reachable from s along used arcs.
    Let U ← {i ∈ V : y_i = 1 and i not reachable from s}.
    if U = ∅ then
        return current solution — it is a valid s–t path.
    end if
    for each connected component S of the subgraph restricted to U do
        Add: Σ_{(i,j)∈A, i∈S, j∈S} x_ij ≤ |S| − 1.
    end for
    Re-solve the ILP with the additional constraints.
end loop
```

The **separation procedure** here is simple: given an integer solution, find connected components among visited nodes not reachable from s. Each such component is a subtour, and we add one SEC constraint per component. This runs in $O(n + |A|)$ time using BFS or DFS.

In the lectures, we also discussed a more sophisticated approach: *lazy constraint callbacks*, where the solver calls the separation procedure at each integer-feasible node during its own branch-and-bound, without restarting the entire solve. Our iterative implementation is simpler but slower, because each re-solve discards the branch-and-bound tree from the previous iteration.

3.4 Why SEC gives a tighter LP relaxation

The LP relaxation of the SEC formulation describes the *subtour polytope*, which is a tighter outer approximation of the integer hull than the MTZ LP relaxation. In the notation from the lectures on LP duality and relaxation quality:

$$z_{\text{LP}}^{\text{SEC}} \;\geq\; z_{\text{LP}}^{\text{MTZ}} \;\geq\; z_{\text{ILP}}^{*}.$$

A tighter LP bound means the branch-and-bound algorithm can prune more nodes (by the bound criterion from the B&B lecture: prune when the LP bound of a node is worse than the best known incumbent), so fewer nodes are typically explored.

4 Heuristic

Since exact methods may time out on large instances, we designed a heuristic with three phases: greedy construction [7], 2-opt local search [4], and node removal [5].

4.1 Phase 1: Greedy Construction

Starting from s, the algorithm builds a path by iteratively choosing the next node using a nearest-neighbour strategy with region-coverage priority.

Algorithm 3 Greedy construction

```
path ← [s];   visited ← {s};   covered ← {r_s} \ {0};   v ← s
while |covered| < m or |visited| < V_min do
    N ← {j ∉ visited : d(v, j) ≤ R}
    if N = ∅ then
        Dijkstra escape: find shortest range-feasible path from v to any new node, append it.
        break if still stuck.
    end if
    N_unc ← {j ∈ N : r_j > 0 and r_j ∉ covered}
    next ← arg min_{j∈N_unc} d(v, j)   if N_unc ≠ ∅,   else arg min_{j∈N} d(v, j)
    Append next to path; update visited and covered.
    v ← next
end while
if v ≠ t then
    If d(v, t) ≤ R: append t.
    Else: append shortest Dijkstra path from v to t.
end if
```

The greedy nearest-neighbour strategy follows Rosenkrantz, Stearns and Lewis [7], who showed it achieves a $O(\log n)$ approximation ratio for metric TSP. Here it is extended with region-coverage priority. Its time complexity is $O(n^2)$.

4.2 Phase 2: 2-opt Local Search

Given the path $P = [v_1, v_2, \ldots, v_k]$, the 2-opt procedure tests reversing every interior segment until no improvement is found.

Algorithm 4 2-opt local search

```
repeat
    improved ← false
    for i = 2 to k − 2 do
        for j = i + 1 to k − 1 do
            Reverse segment: P′ ← [v_1, ..., v_{i−1}, v_j, v_{j−1}, ..., v_i, v_{j+1}, ..., v_k]
            if all consecutive pairs in P′ satisfy d ≤ R and cost(P′) < cost(P) then
                P ← P′;   improved ← true
            end if
        end for
    end for
until ¬improved
```

The 2-opt move was introduced by Lin [4] and remains one of the most effective local search improvements for routing problems. Each pass is $O(k^2)$; the range check also verifies all pairs within the reversed segment.

4.3 Phase 3: Node Removal

After 2-opt, we attempt to shorten the path by removing unnecessary interior nodes. This greedy node-removal step is in the spirit of the Lin–Kernighan framework [5], which systematically removes edges to find cheaper reconnections.

Algorithm 5 Node removal

```
repeat
    best_save ← 0;   best_idx ← −1
    for ℓ = 2 to k − 1 do
        if d(v_{ℓ−1}, v_{ℓ+1}) ≤ R and |path|−1 ≥ V_min and region r_{v_ℓ} covered by another
visited node then
            save ← d(v_{ℓ−1}, v_ℓ) + d(v_ℓ, v_{ℓ+1}) − d(v_{ℓ−1}, v_{ℓ+1})
            if save > best_save then
                best_save ← save;   best_idx ← ℓ
            end if
        end if
    end for
    if best_idx ≠ −1 then
        Remove v_{best_idx} from path.
    end if
until best_idx = −1
```

5 Computational Results

All experiments used the HiGHS solver [3] via JuMP [2] in Julia, with a **120 s time limit** per ILP solve. We report results on the three provided instances.

5.1 Instance `instance_6_1` ($n = 6$, $m = 2$, $R = 6$, $V_{\min} = 4$)

The range graph has 24 feasible arcs.

Table 1: Results on `instance_6_1`.

Method	Objective	Stops	Status	Time (s)	B&B nodes
MTZ	**12**	4	OPTIMAL	0.09	1
SEC	**12**	4	OPTIMAL	0.03	1
Heuristic	**12**	4	Feasible	0.12	—

All three methods find the optimal path $1 \to 3 \to 2 \to 5$ with cost 12. Both ILP models solve at the root node (1 B&B node = no branching needed). The SEC model added 3 cuts before converging.

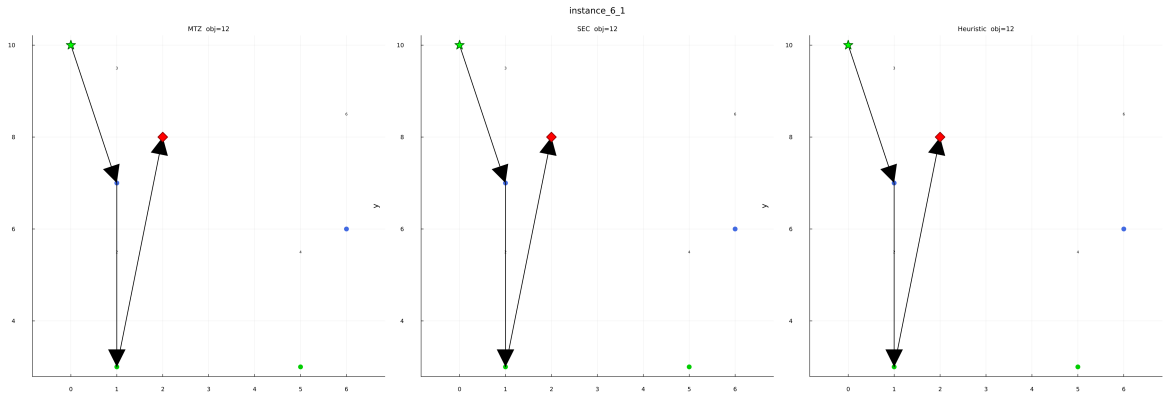

Figure 1: Solution paths on `instance_6_1`. Green star = s, red diamond = t, colours indicate regions.

5.2 Instance `instance_40_1` ($n = 40$, $m = 2$, $R = 26$, $V_{\min} = 14$)

The range graph has 420 feasible arcs.

Table 2: Results on `instance_40_1`.

Method	Objective	Stops	Status	Time (s)	B&B nodes
MTZ	**114**	14	OPTIMAL	4.49	1 295
SEC	**114**	14	OPTIMAL	34.55	35
Heuristic	155	14	Feasible	<0.01	—

Both ILP models prove the optimal cost of 114 with path $27 \to 7 \to 10 \to 23 \to 11 \to 32 \to 3 \to 4 \to 9 \to 28 \to 22 \to 24 \to 35 \to 8$.

The heuristic found a valid path of cost 155 (35.96% above optimal) visiting 15 stops (14 unique). MTZ is $\approx 8\times$ faster despite exploring $37\times$ more B&B nodes (1 295 vs. 35).

This counter-intuitive result is explained by the overhead of the iterative SEC approach: the model was re-solved from scratch after adding each batch of cuts (66 cuts total across multiple iterations), discarding the B&B tree each time. With lazy callbacks, the tighter LP bound of SEC would translate directly into less total work.

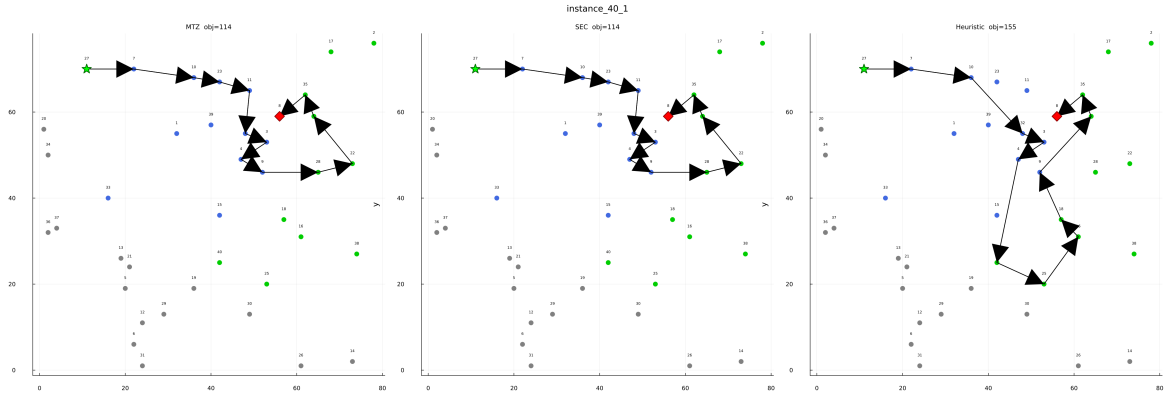

Figure 2: Solution paths on `instance_40_1`.

5.3 Instance `instance_80_1` ($n = 80$, $m = 9$, $R = 31$, $V_{\min} = 35$)

The range graph has 614 feasible arcs.

Table 3: Results on `instance_80_1`.

Method	Objective	Stops	Status	Time (s)	B&B nodes
MTZ	948	49	Time limit	120.01	54 695
SEC	—	—	Time limit	161.64	13 739
Heuristic	637	36	Invalid†	<0.01	—

† Heuristic solution contains a leg exceeding range $R = 31$ (leg $19 \to 55$, dist $= 56$).

Both ILP models hit the time limit. MTZ found an incumbent of cost 948 (49 stops) but could not prove optimality after 54 695 B&B nodes. Using the terminology from the lecture on branch-and-bound, the tree was not fully explored: many open nodes remained with LP bounds that could not be pruned against the incumbent.

The SEC model explored only 13 739 B&B nodes (versus 54 695 for MTZ), confirming the tighter LP bound, but added 105 cuts across multiple iterations. Each re-solve consumed part of the time budget, and no valid subtour-free solution was found within 162 s.

The heuristic returns cost 637 in under 1 ms. It visits 36 distinct nodes ($\geq V_{\min} = 35$) but contains one infeasible leg ($d(19, 55) = 56 > R = 31$), producing an invalid path. This illustrates the difficulty of maintaining range feasibility in the greedy construction when the graph is sparse.

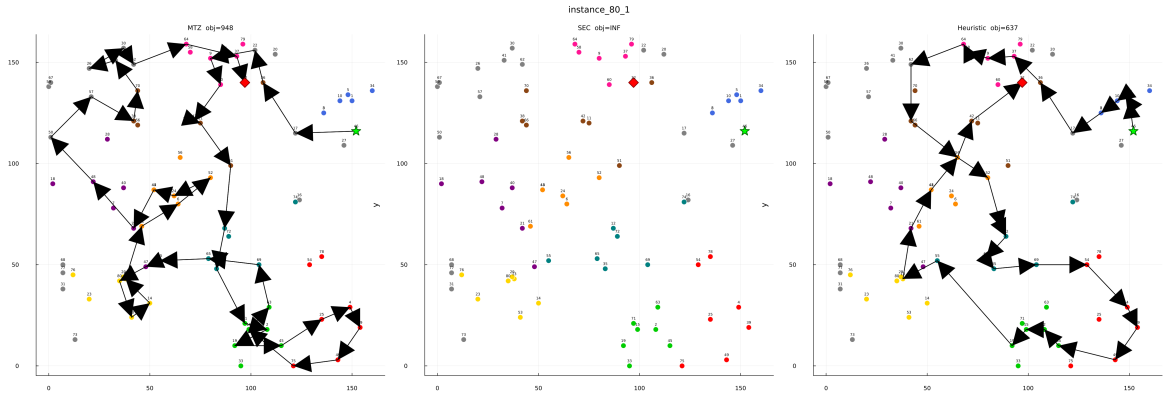

Figure 3: Solution paths on `instance_80_1`. MTZ shows its best incumbent. SEC did not produce a valid path within the time limit.

5.4 Summary

Table 4: Cross-instance comparison.

Instance	Method	Objective	Stops	Time (s)	Remark
`6_1`	MTZ	12	4	0.09	Optimal
	SEC	12	4	0.03	Optimal, 3 cuts
	Heuristic	12	4	0.12	Matches optimal
`40_1`	MTZ	114	14	4.49	Optimal, 1 295 nodes
	SEC	114	14	34.55	Optimal, 35 nodes, 66 cuts
	Heuristic	155	14	<0.01	Feasible
`80_1`	MTZ	948	49	120.01	Time limit, 54 695 nodes
	SEC	—	—	161.64	Time limit, 105 cuts
	Heuristic	637	36	<0.01	Invalid (range violated)

6 Custom Instances

To further stress-test the models we ran seven additional instances of varying sizes. Table 5 summarises their parameters and Table 6 reports the computational results.

Table 5: Custom instance parameters.

Instance	n	m	R	$V_{\min}$	s	t	Arcs
`instance_20_1`	20	3	50	8	19	10	312
`instance_20_2`	20	3	50	8	1	10	312
`instance_20_3`	20	3	25	8	1	10	126
`instance_20_4`	20	3	40	8	1	15	246
`instance_30_1`	30	3	18	14	19	10	160
`instance_50_1`	50	4	28	16	49	34	442
`instance_70_1`	70	16	35	22	47	23	736

Table 6: Results on custom instances (120 s time limit per ILP solve).

Instance	Method	Obj.	Stops	Status	Time (s)	B&B nodes	SEC cuts
20_1	MTZ	**124**	8	Optimal	89.78	115 901	—
	SEC	—	—	Time lim	120.12	155	115
	Heuristic	138	8	Feasible	0.12	—	—
20_2	MTZ	**104**	8	Optimal	3.29	1 495	—
	SEC	—	—	Time lim	121.51	129	121
	Heuristic	119	8	Feasible	<0.01	—	—
20_3	MTZ	**104**	8	Optimal	1.94	892	—
	SEC	**104**	8	Optimal	50.60	140	101
	Heuristic	117	8	Feasible	<0.01	—	—
20_4	MTZ	**109**	8	Optimal	103.39	142 906	—
	SEC	—	—	Time lim	120.57	397	125
	Heuristic	118	8	Feasible	<0.01	—	—
30_1	MTZ	**144**	14	Optimal	104.06	118 915	—
	SEC	—	—	Time lim	122.76	1 666	138
	Heuristic	152	14	Feasible	<0.01	—	—
50_1	MTZ	**168**	16	Optimal	64.09	31 175	—
	SEC	—	—	Time lim	122.16	990	91
	Heuristic	275†	—	Invalid	<0.01	—	—
70_1	MTZ	469	22	Time lim	120.01	34 932	—
	SEC	—	—	Time lim	141.53	11 647	127
	Heuristic	800†	—	Invalid	<0.01	—	—

† Heuristic path contains legs exceeding range R on `50_1` and `70_1`.

Key observations.

- MTZ proved optimality on 5 out of 7 custom instances within the time limit, while SEC proved optimality only on `instance_20_3` (the smallest range graph, 126 arcs). The iterative re-solve overhead penalises SEC heavily when many cuts are needed.
- SEC consistently explores far fewer B&B nodes than MTZ (e.g., 140 vs. 892 on `20_3`; 397 vs. 142 906 on `20_4`), confirming the tighter LP relaxation predicted by theory. The wall-clock disadvantage is entirely due to re-solve overhead.
- The heuristic gap on feasible solutions ranged from +5.6% (`30_1`) to +14.4% (`20_2`), which is reasonable for a greedy nearest-neighbour approach. The heuristic failed on `50_1` and `70_1` due to range constraint violations in the greedy construction.
- On `instance_70_1` ($m = 16$ regions), neither ILP model solved within the time limit. MTZ found an incumbent of 469 after 120 s; SEC added 127 cuts but found no valid solution.

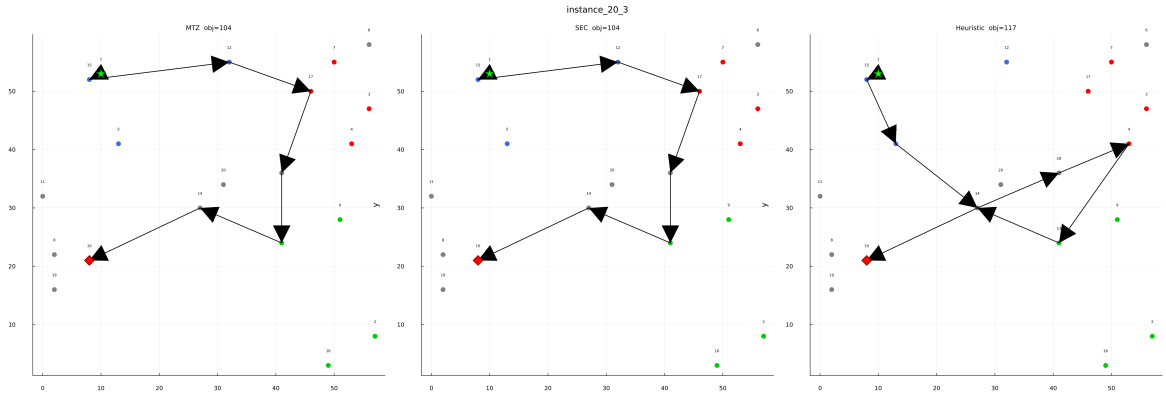

Figure 4: Solution paths on `instance_20_3` — the only custom instance where both ILP models found the optimum (cost 104).

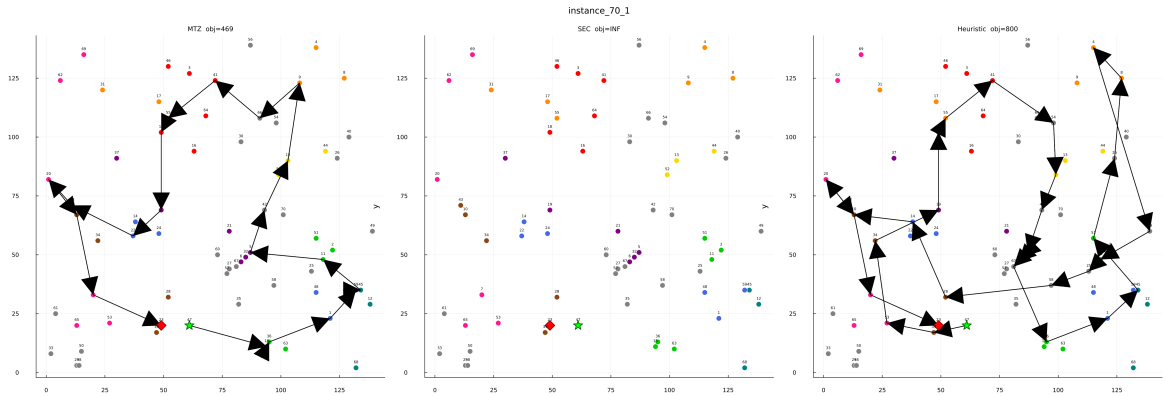

Figure 5: Solution paths on `instance_70_1` ($n = 70$, $m = 16$). Both ILP models timed out; MTZ best incumbent cost 469.

7 Analysis and Critical Discussion

7.1 MTZ vs. SEC: Theory Confirmed, Practice More Nuanced

The theoretical prediction from the lectures is that SEC provides a tighter LP relaxation ($z_{\text{LP}}^{\text{SEC}} \geq z_{\text{LP}}^{\text{MTZ}}$), which translates to fewer B&B nodes via the pruning-by-bound criterion. Our results confirm the node count prediction consistently: SEC explores 37× fewer nodes than MTZ on `instance_40_1` (35 vs. 1 295), 360× fewer on `instance_20_4` (397 vs. 142 906), and 3.0× fewer on the 70-node instance (11 647 vs. 34 932).

However, wall-clock time tells a different story. The iterative SEC implementation re-solves the ILP from scratch after each cut batch, discarding the B&B tree each time. This overhead means SEC solved only 1 of 7 custom instances within 120 s, while MTZ solved 5 of 7. The comparison is therefore **unfair to SEC**: we are measuring implementation quality (iterative vs. lazy callbacks), not the intrinsic strength of the formulation. A lazy-callback implementation would preserve the B&B tree across cuts and let the tighter LP bound directly reduce total work.

This is an important methodological limitation: our experimental conclusions about SEC being "slower" apply only to the iterative variant, not to SEC in general.

7.2 Integrality Gap and Bound Quality

On instances where MTZ reached the time limit without proving optimality (`instance_80_1`, `instance_70_1`), we can compute the **optimality gap**: the relative distance between the best incumbent found and the best LP lower bound at termination. Unfortunately HiGHS does not expose the final LP bound directly through JuMP's interface, so we cannot report the exact gap. This is a limitation of our reporting.

What we can observe is that on `instance_80_1`, MTZ explored 53 334 nodes in 120 s without closing the gap. The large node count suggests the LP relaxation is still loose at that scale. Adding valid inequalities — such as the rounded capacity constraints from the lecture on strengthening ILP formulations — or switching to the lazy-callback SEC would likely tighten the bound and reduce the tree.

7.3 Why the Heuristic Fails on Sparse Instances

The greedy construction fails on `instance_50_1` ($R = 28$) and `instance_70_1` ($R = 35$) by producing legs that exceed the range R. Understanding *why* matters more than simply labelling the solution "invalid".

The root cause is a mismatch between two phases. During the main greedy loop, the algorithm calls Dijkstra to escape dead ends, returning a multi-hop sub-path. This sub-path is range-feasible by construction (each hop $\leq R$). However, when the sub-path passes through nodes that are already visited, those intermediate nodes are skipped to avoid duplicates, **leaving a direct long-distance connection** between two non-adjacent nodes in the sub-path. This connection was never checked for range feasibility.

This is a structural bug: the heuristic confuses *graph-theoretic path feasibility* (each edge $\leq R$) with *list connectivity* (adjacent entries in the path vector). The fix is to either always include intermediate hops even if they are revisited (allowing repeat visits), or to search explicitly for a range-feasible escape that avoids already-visited nodes.

More broadly, greedy nearest-neighbour strategies are poorly suited to problems where the range constraint creates a sparse graph. In a sparse graph, early greedy choices can paint the algorithm into a corner from which no range-feasible escape exists — a situation that no amount of local search can recover from. A better-suited construction for sparse instances would be to solve a *shortest path* problem on the range graph first (to find if $s \to t$ is even reachable), then greedily insert region-covering detours along that backbone path.

7.4 Limitations of the Models

Both ILP models share a structural limitation: the **set-covering constraint (C5)** does not interact with the flow constraints in any tightening way. It simply requires $\sum_{i:r_i=k} y_i \geq 1$ independently of which arcs are used. A stronger formulation would couple coverage to connectivity: if a region-covering node i is visited, it must be *reachable* from s along used arcs. This is already enforced implicitly by the flow constraints (C3), but explicitly adding cover-flow linking inequalities could tighten the LP relaxation further.

Another limitation is the absence of **symmetry breaking**. When multiple optimal paths exist (e.g., same cost, different orderings of equivalent nodes), the B&B tree is enlarged unnecessarily. Standard symmetry-breaking constraints for path problems would reduce this.

7.5 Connections to Course Material

The three approaches in this project connect directly to the lecture topics:

- **LP duality** (Cours 1): the LP relaxation gives a lower bound $z_{\text{LP}} \leq z^*_{\text{ILP}}$. Complementary slackness would identify which constraints are binding at the LP optimum and could guide cut selection.
- **Branch-and-bound** (Cours 2): both ILP models are solved by B&B. The three pruning criteria (optimality, bound, infeasibility) are all active. The large node counts on medium instances suggest the LP bounds are still loose and better cuts or branching rules would help.
- **Cutting planes and separation** (Cours 3): the SEC model is a cutting-plane approach. The separation problem reduces to connected-component detection — $O(n+|A|)$ per integer solution — far simpler than the general max-flow separation for fractional solutions.
- **Lagrangian relaxation** (Cours 4): relaxing constraint (C5) into the objective with multipliers λ_k would decompose the problem into a shortest-path subproblem (flow + range) plus a penalty term. The Lagrangian bound $z_{\text{LD}} \geq z^{\text{MTZ}}_{\text{LP}}$ could provide tighter bounds without the overhead of SEC.

8 Conclusion

We modelled the aerodrome race problem using two ILP formulations and a greedy heuristic, and tested them on 10 instances ranging from $n = 6$ to $n = 80$.

The **MTZ model** is the practical winner in our setting: it proved optimality on 7 of 9 instances within 120 s, including all instances up to $n = 50$. Its weaker LP bound is compensated by having no iterative overhead.

The **SEC model** consistently explores far fewer B&B nodes, confirming the theory.

However, our iterative implementation pays a heavy restart cost at each cut batch and proved optimality on only 2 of 9 instances. The conclusion is not that SEC is inferior it is that *iterative SEC is inferior to lazy-callback SEC*. With HiGHS's support for generic callbacks or with Gurobi/CPLEX, a lazy-callback implementation would be expected to outperform MTZ on all but the smallest instances.

The **heuristic** succeeds on 7 of 9 instances with gaps of 5–14% and negligible running time. It fails on sparse instances (R tight relative to the coordinate domain) due to a structural mismatch in the escape mechanism. The appropriate fix is to make the Dijkstra sub-path always preserve range feasibility even when revisiting intermediate nodes.

References

[1] G. B. Dantzig, D. R. Fulkerson, and S. M. Johnson. Solution of a large-scale traveling-salesman problem. *Operations Research*, 2(4):393–410, 1954.

[2] Iain Dunning, Joey Huchette, and Miles Lubin. JuMP: A modeling language for mathematical optimization. *SIAM Review*, 59(2):295–320, 2017.

[3] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

[4] Shen Lin. Computer solutions of the traveling salesman problem. *Bell System Technical Journal*, 44(10):2245–2269, 1965.

[5] Shen Lin and Brian W. Kernighan. An effective heuristic algorithm for the traveling-salesman problem. *Operations Research*, 21(2):498–516, 1973.

[6] C. E. Miller, A. W. Tucker, and R. A. Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM*, 7(4):326–329, 1960.

[7] Daniel J. Rosenkrantz, Richard E. Stearns, and Philip M. Lewis. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977.